

HAGE: Harnessing Agentic Memory via RL-Driven Weighted Graph Evolution

Dongming Jiang^α, Yi Li^α, Guanpeng Li^β, Qiannan Li^γ, Bingzhe Li^{α*}

^αDepartment of Computer Science, The University of Texas at Dallas

^βDepartment of Electrical and Computer Engineering, University of Florida

^γUniversity of California, Davis

{dongming.jiang, yi.li3, bingzhe.li}@utdallas.edu

liguanpeng@ufl.edu

qnli@ucdavis.edu

Abstract

Memory retrieval in agentic large language model (LLM) systems is often treated as a static lookup problem, relying on flat vector search or fixed binary relational graphs. However, fixed graph structures cannot capture the varying strength, confidence, and query-dependent relevance of relationships between events. In this paper, we propose HAGE, a weighted multi-relational memory framework that reconceptualizes retrieval as sequential, query-conditioned traversal over a unified relational memory graph. Memory is organized as relation-specific graph views over shared memory nodes, where each edge is associated with a trainable relation feature vector encoding multiple relational signals. Given a query, an LLM-based classifier identifies the relational intent, and a routing network dynamically modulates the corresponding dimensions of the edge embedding. Traversal scores are computed via a learned combination of semantic similarity and these query-conditioned edge representations. This allows memory traversal to prioritize high-utility relational paths while softly suppressing noisy or weakly relevant connections. Beyond adaptive traversal, HAGE further introduces a reinforcement learning-based training framework that jointly optimizes routing behavior and edge representations using downstream tasks. Finally, empirical results demonstrate improved long-horizon reasoning accuracy and a favorable accuracy-efficiency trade-off compared to state-of-the-art agentic memory systems.

1 Introduction

Large Language Models (LLMs) have rapidly become the foundation of modern AI agents (Brown et al., 2020a; Achiam et al., 2023; Wei et al., 2022a; Yao et al., 2022; Shinn et al., 2023; Park et al., 2023), enabling strong performance in reasoning, planning, tool use, and multi-turn inter-

action (Brown et al., 2020b; Achiam et al., 2023; Wei et al., 2022b). However, effective agency requires more than solving isolated prompts. A long-horizon agent must accumulate experience, retain user- and task-specific information, and selectively reuse past evidence across sessions. This requirement exposes a fundamental limitation of context-only interaction: even when long-context models are available, relevant information can be diluted, misplaced, or forgotten as interactions grow, leading to unstable recall and degraded long-term reasoning (Liu et al., 2024; Beltagy et al., 2020a; Maharana et al., 2024; Wu et al., 2024).

Retrieval-Augmented Generation (RAG) and memory-augmented generation systems address this issue by moving part of the agent’s knowledge outside the model parameters and into an explicit, queryable memory store (Lewis et al., 2020; Borgeaud et al., 2022; Packer et al., 2024; Zhong et al., 2024). Such external memories allow agents to preserve information beyond the current context window, support multi-session continuity, and adapt responses based on accumulated experience. Recent agent-memory systems further move beyond simple document retrieval by extracting salient memories, updating them over time, and organizing them into structured representations such as episodic records, semantic summaries, entity-centric memories, or graph-based links (Xu et al., 2025; Chhikara et al., 2025). These designs show that the structure of memory is crucial for long-term agent behavior.

Despite this progress in structuring memory, a central challenge remains underexplored: *how should an agent prioritize and navigate these complex connections?* Graph-based memory and graph-augmented retrieval have emerged as promising directions for capturing semantic, temporal, causal, and entity-centric dependencies in complex reasoning tasks (Edge et al., 2024; Gutiérrez et al., 2024; Rasmussen et al., 2025; Anokhin et al.,

*Corresponding author

2024). However, most existing agent-memory approaches still rely on unweighted or weakly weighted relations, where an edge primarily indicates the existence of a connection rather than its query-dependent utility. This is a critical bottleneck. In real-world reasoning, the importance of a connection is inherently query-dependent. For example, a temporal edge might be essential for answering a sequence-based question but irrelevant for an entity-centric query. By treating outgoing connections as equally valid or using fixed graph-expansion rules, existing systems can fail to discriminate between highly relevant pathways and distracting noise, leading to degraded retrieval accuracy as memory grows.

Furthermore, even when continuous scores or edge weights are introduced, retrieval is still largely governed by fixed similarity search, manually designed scoring functions, or static heuristic traversal rules. Recent work on adaptive RAG and graph-based retrieval suggests that retrieval decisions can be optimized through learned policies or reinforcement learning rather than predefined pipelines (Guo et al., 2025; Yu et al., 2026a). However, these methods mainly target external knowledge-intensive QA or text-graph hybrid retrieval, rather than persistent agentic memory where the memory graph evolves across interactions. This gap motivates a shift toward dynamic routing for agentic memory: instead of relying on handcrafted access mechanisms, an agent should learn which relational paths to follow based on the immediate query and downstream feedback.

To address these limitations, we propose HAGE, a weighted multi-relational memory framework that reconceptualizes memory retrieval as query-conditioned traversal over a multi-relational memory graph with relation-specific views, trained with reinforcement learning-based optimization. HAGE is built on two key principles.

First, memory is structured as a family of relation-specific graphs with trainable edge embeddings. Instead of static scalar weights, each embedding encodes multiple relational dimensions. Given a query, an LLM-based classifier identifies the relational intent, and a routing network dynamically modulates these edge features. By additively combining semantic similarity with this query-conditioned structural weight, the system respects both content relevance and structural alignment. This design enables query-dependent routing, allowing the agent to efficiently traverse structurally

critical but semantically distant bridge nodes.

Second, HAGE introduces a reinforcement learning-based training framework for adaptive retrieval. Instead of relying on fixed traversal heuristics, the model learns to optimize relation-aware routing behavior using downstream task feedback. In our formulation, trainable edge representations capture which relational connections are useful for different query types, while the routing component determines how retrieval proceeds conditioned on the query. This coupling allows the retrieval policy and memory representations to be optimized jointly, yielding a learned alternative to handcrafted graph traversal strategies.

Together, these contributions shift agentic memory from fixed heuristic retrieval toward learned relation-aware retrieval. Instead of relying solely on manually designed graph scoring rules, HAGE treats retrieval as an optimized, query-conditioned traversal process over a multi-relational memory graph.

Our contributions are summarized as follows:

1. A weighted multi-relational memory architecture in which a multi-relational memory graph is augmented with learnable edge representations, enabling fine-grained, per-edge discrimination beyond static or type-level heuristic scoring.
2. A reinforcement learning framework that formulates query-conditioned graph retrieval as a sequential decision process. It jointly optimizes routing behavior and edge representations using downstream task feedback, requiring only node-level evidence targets rather than full path-level trajectory supervision.
3. An empirical analysis showing that joint optimization with regularization improves generalization over routing-only and edge-only variants, highlighting the importance of learned edge representations for robust graph-based memory retrieval.¹

2 Background

2.1 From Static Retrieval to Agentic Memory

Retrieval-Augmented Generation (RAG) improves language models by retrieving relevant information from an external datastore and conditioning generation on the retrieved context (Lewis et al., 2020).

¹The anonymized code repository is available at: https://anonymous.4open.science/r/HAGE_review-C048/README.md.

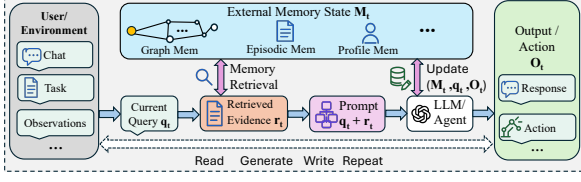


Figure 1: High-Level Architecture of Memory-Augmented Generation (MAG).

While this paradigm is effective for relatively static corpora, long-horizon agents require a more dynamic form of retrieval: they must accumulate, update, and reuse information generated through their own interactions. This motivates Memory-Augmented Generation (MAG) as shown in Figure 1, where the memory store is not only queried but also revised over time as the agent observes new events, user preferences, task outcomes, and environmental feedback (Park et al., 2023; Packer et al., 2024; Nan et al., 2025; Chhikara et al., 2025; Xu et al., 2025).

Formally, at interaction step t , an agent maintains a mutable memory state $\mathcal{M}_t$. Given a query or observation q_t , the agent retrieves relevant evidence from memory, generates an output, and then updates the memory state:

$$r_t = \text{Retrieve}(q_t, \mathcal{M}_t), \quad (1)$$

$$o_t = \text{LLM}(q_t, r_t), \quad (2)$$

$$\mathcal{M}_{t+1} = \text{Update}(\mathcal{M}_t, q_t, o_t). \quad (3)$$

This read–generate–write loop distinguishes agentic memory from conventional retrieval. The memory system must not only preserve useful information, but also determine how relevant evidence should be accessed.

Recent work has explored increasingly structured forms of agent memory, including episodic summaries, note-like memory units, entity-centered memory stores, and graph-based relational memories (Liu et al., 2023; Xu et al., 2025; Nan et al., 2025; Edge et al., 2024; Rasmussen et al., 2025; Kiciman et al., 2023). Graph-based memory is particularly appealing because it can encode semantic, temporal, causal, and entity relations explicitly, allowing retrieval to exploit relational structure instead of relying only on embedding similarity. However, in many such systems, memory access still depends on fixed edge types, manually designed weighting rules, or heuristic traversal procedures. Thus, although the memory representation

becomes more expressive, the access mechanism often remains static.

2.2 Learning Memory Access as Sequential Decision Making

HAGE focuses on this underexplored problem: how to learn the retrieval behavior of a structured memory system. We view graph-based memory access as a sequential decision process. Given a query and the current memory graph, the system must decide which neighbors to expand, which relational cues to emphasize, and which memory nodes to include in the retrieved context. This formulation is particularly natural for multi-hop, temporal, and causal queries, where the usefulness of a memory item depends not only on its individual relevance but also on the path through which it is reached.

This perspective connects graph-based memory retrieval with reinforcement learning. Rather than treating traversal as a fixed procedure, one can optimize retrieval decisions using rewards derived from downstream evidence quality. HAGE adopts this view by making both edge representations and routing behavior trainable. Edge features capture relation-aware traversal preferences, while the routing policy learns how to traverse the graph under task-level feedback. In this way, memory structure and memory access are optimized jointly rather than designed independently.

3 HAGE Design

In this section, we introduce HAGE, a framework that reconceptualizes memory retrieval in agentic systems as sequential, query-conditioned traversal over structured relational memory, rather than as static lookup. HAGE consists of two key components: (1) a weighted multi-relational graph memory for capturing heterogeneous and strength-sensitive relations among memory events, and (2) a reinforcement learning-based training framework for jointly optimizing relation-aware retrieval policies and edge representations. We first present the construction of the weighted graph memory and its query-conditioned traversal mechanism, followed by the learning framework used to optimize routing behavior and relational edge weights.

3.1 Overview

HAGE is built on the insight that memory retrieval in agentic systems requires more than static lookup: it often involves sequential, query-conditioned

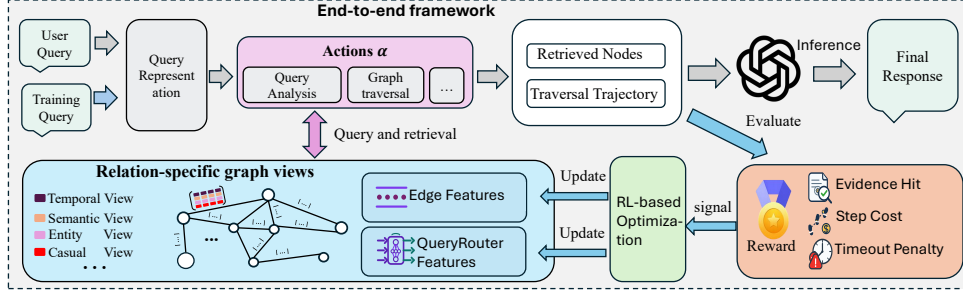


Figure 2: Architectural Overview of HAGE.

traversal over structured memory. To operationalize this perspective, HAGE integrates two tightly coupled components, as illustrated in Figure 2.

- A **weighted multi-relational memory graph**, where each edge carries a trainable feature vector encoding relation-aware traversal preferences. These features are initialized from a heuristic scoring phase and refined through downstream reward signals.
- A **reinforcement learning-based training framework** that jointly optimizes a query-conditioned routing network and the edge representations using policy-gradient updates.

Unlike prior graph-based memory systems that rely on fixed edge types and hand-designed scoring rules, HAGE makes relation weighting query-adaptive and learnable.

3.2 Weighted Multi-Relational Memory Graph

We represent memory as a directed multigraph $\mathcal{G}_t = (\mathcal{N}_t, \mathcal{E}_t)$. The edge set is decomposed into four relation-specific subsets that capture temporal adjacency, semantic similarity, causal dependence, and entity co-reference:

$$\mathcal{E}_t = \mathcal{E}_{temp} \cup \mathcal{E}_{sem} \cup \mathcal{E}_{causal} \cup \mathcal{E}_{ent}. \quad (4)$$

Nodes are hierarchically organized into fine-grained *Event-Nodes*. Each Event-Node n_i is represented as

$$n_i = \langle c_i, \tau_i, \mathbf{v}_i, \mathcal{A}_i \rangle, \quad (5)$$

where c_i denotes the event content, τ_i is the associated timestamp, $\mathbf{v}_i \in \mathbb{R}^d$ is a dense semantic embedding, and $\mathcal{A}_i$ contains structured metadata associated with the event.

A key design choice in HAGE is that each edge (i, j) is associated with a trainable relation feature vector $\mathbf{e}_{ij} \in \mathbb{R}^R$, where $R = 4$ in this design,

corresponding to temporal, semantic, causal, and entity-based relations. When an LLM-based edge-scoring cache is available, we initialize this vector as

$$\mathbf{e}_{ij}^{(0)} = [s_{temp}, s_{sem}, s_{causal}, s_{ent}]^\top, \quad (6)$$

where s_r denotes the initial score assigned to relation type r . In the absence of cached scores, $\mathbf{e}_{ij}^{(0)}$ is initialized as a one-hot vector corresponding to the edge’s primary relation type. During training, these edge features are optimized as learnable parameters and updated using downstream reward signals.

3.3 Query-Conditioned Retrieval

Given a query q and graph $\mathcal{G}_t$, HAGE performs retrieval in four stages: query analysis, anchor identification, weighted traversal, and context synthesis.

Query analysis and anchor identification. The query is mapped to structured control signals, including a relation intent T_q , a dense embedding $\vec{q}$, and auxiliary lexical or temporal constraints when available. To initialize traversal robustly, the system identifies anchor nodes by fusing multiple retrieval signals, including dense vector retrieval, sparse lexical matching, and temporal filtering. In practice, this stage provides reliable entry points, while the core contribution of HAGE lies in the learned traversal that follows.

Query-conditioned weighted traversal. Starting from the anchor set $\mathcal{S}_{anchor}$, the system expands the retrieved context through weighted graph traversal. For a given query q , let $\mathbf{v}_{T_q}$ denote the dense embedding of the relation intent T_q identified by the LLM-based classifier. For each edge (i, j) , the static feature $\mathbf{e}_{ij}$ is augmented with runtime similarity features and the query intent:

$$\tilde{\mathbf{e}}_{ij} = [\mathbf{e}_{ij}; \mathbf{v}_{T_q}; \cos(\vec{q}, \mathbf{v}_i); \cos(\vec{q}, \mathbf{v}_j)]. \quad (7)$$

The enriched feature and query embedding are passed through a lightweight MLP, denoted *QueryRouter*, which produces a positive scalar structural weight:

$$w_{ij}(q) = \text{softplus}(\text{MLP}([\vec{q}; \tilde{\mathbf{e}}_{ij}])). \quad (8)$$

To ensure the agent can traverse structurally critical but semantically distant “bridge” nodes, the final transition score is defined as an additive combination of semantic relevance and the learned structural weight:

$$S(n_j | n_i, q) = \lambda \cos(\mathbf{v}_j, \vec{q}) + (1 - \lambda)w_{ij}(q), \quad (9)$$

where $\lambda \in [0, 1]$ is a balancing hyperparameter. This additive form ensures that an edge can be strongly preferred if it possesses high structural importance, even if the target node has a negative semantic cosine similarity. The resulting traversal policy is

$$\pi(n_j | n_i, q) = \frac{\exp(S(n_j | n_i, q))}{\sum_{n_k \in \mathcal{N}(n_i)} \exp(S(n_k | n_i, q))}, \quad (10)$$

where $\mathcal{N}(n_i)$ denotes the neighbors of n_i . During training, actions are sampled from π for exploration; at inference time, the system uses greedy selection or beam-style expansion over high-scoring candidates. Traversal terminates when the hop budget is exhausted or target evidence is reached.

Context synthesis. The retrieved nodes are re-ordered and serialized into a compact context for the downstream LLM. Depending on query type, nodes are organized temporally, causally, or by retrieval score, and are included until the context budget is exhausted.

3.4 Reinforcement Learning-Based Joint Optimization

HAGE optimizes relation-aware retrieval by formulating graph traversal as a Markov Decision Process (MDP) and training the routing network and edge representations jointly via policy gradient methods.

MDP Formulation. Each training example defines a per-query episode:

- **State:** The current node n_i , the query embedding $\vec{q}$, and a visited-node mask $\mathcal{V}_t$ to prevent cyclic loops.
- **Action:** Selecting a neighbor $n_j \in \mathcal{N}(n_i)$ according to the stochastic policy $\pi_\theta(n_j | n_i, q)$.

- **Transition:** The agent moves to n_j and the step count increments.
- **Termination:** The episode ends when the agent reaches a target evidence node, encounters a dead end (no unvisited neighbors), or exhausts the hop budget H_{max} .

The start node is selected as the node with highest cosine similarity to the query embedding, simulating the anchor identification stage during training.

Reward Design. The reward combines an evidence-hit signal with shaping penalties for traversal cost:

$$r_t = r_t^{hit} - \lambda_{step} r_t^{step} - \lambda_{timeout} r_t^{timeout}, \quad (11)$$

where r_t^{hit} rewards retrieving target evidence nodes (identified during training by matching node content with ground-truth answers). For multi-hop queries, the agent accumulates r_t^{hit} for each unique target found; traversal terminates only when all required evidence is collected, a dead end is reached, or the hop budget is exhausted. Lastly, r_t^{step} and $r_t^{timeout}$ penalize excessive hops and budget exhaustion, encouraging the model to discover efficient, direct relational paths.

Policy Gradient with Baseline Subtraction. We optimize the traversal policy using REINFORCE with an exponential moving average baseline for variance reduction. For a trajectory $\tau = (n_0, a_0, r_0, \dots, n_T)$, the discounted return at step t is

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}, \quad (12)$$

where γ is the discount factor. The policy-gradient update is

$$\nabla_\theta \mathcal{J} = \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot (G_t - b), \quad (13)$$

where b is a running baseline updated using exponential moving averaging. The parameter set θ includes both the QueryRouter weights and the trainable edge features, allowing the two components to be optimized under the same reward signal. Gradients are clipped to improve stability.

Anchor Regularization. Since the edge features are warm-started from Phase 1 scores, unconstrained optimization may cause them to drift far from their initial values. This creates a distribution mismatch at inference: unseen graphs use static

Phase 1 features, while the router was trained on drifted features. To prevent this, we add an L2 anchor regularization term:

$$\mathcal{L}_{anchor} = \lambda_{anchor} \sum_{(i,j) \in \mathcal{E}_{train}} \left\| \mathbf{e}_{ij} - \mathbf{e}_{ij}^{(0)} \right\|_2^2, \quad (14)$$

where $\mathbf{e}_{ij}^{(0)}$ denotes the frozen Phase 1 initialization. The total training objective combines the policy gradient with this regularization:

$$\mathcal{L} = -\mathcal{J}(\theta) + \mathcal{L}_{anchor}. \quad (15)$$

This formulation can be interpreted as a form of constrained policy learning, where exploration in the feature space is explicitly regularized toward a semantically meaningful initialization, enabling robust generalization to new memory graphs.

3.4.1 Co-Evolutionary Training Dynamics

The joint optimization creates a co-evolutionary dynamic between two parameter groups:

- **Edge features** ($\mathbf{e}_{ij}$) adapt to encode traversal-relevant signals that the router can exploit. Features on successful trajectories are reinforced, while those on unsuccessful paths are suppressed.
- **QueryRouter weights** learn to map query–edge feature pairs to traversal preferences, discovering which feature patterns predict useful transitions for different query types.

To stabilize this feedback-driven co-evolution, we use asymmetric learning rates: η_{router} for the QueryRouter and $\eta_{edge} < \eta_{router}$ for the edge features. This allows the router to adapt rapidly to query-conditioned traversal preferences, while edge features evolve more conservatively to preserve the Phase 1 semantic structure and avoid unstable feature drift.

3.5 Implementation

HAGE is implemented in PyTorch as a modular graph-based training framework. Each memory graph is represented using node embeddings, COO-format edge indices, typed edge labels, and relation-specific edge features, enabling GPU-accelerated routing and edge optimization. We use all-MiniLM-L6-v2 (Reimers and Gurevych, 2019) to initialize node embeddings and precompute adjacency lists for efficient traversal.

Training is performed with sample-level cross-validation. The router and edge modules are optimized with Adam (Kingma and Ba, 2014), using

separate learning rates for routing and edge-feature updates. The best checkpoint is selected based on validation routing success rate. Importantly, Phase 2 training requires no LLM calls, operating only on cached graph structures and pre-computed embeddings. Detailed hyperparameters are provided in Appendix B.

4 Experiments

We conduct comprehensive experiments to evaluate the proposed HAGE architecture, focusing on three aspects: (1) end-to-end reasoning accuracy on long-term memory benchmarks, (2) the effectiveness of co-evolutionary edge learning via ablation studies, and (3) system efficiency under realistic deployment conditions.

4.1 Experimental Setup

Datasets. We evaluate memory retrieval capability using two widely adopted benchmarks: (1) **Lo-CoMo** (Maharana et al., 2024): which contains ultra-long conversations (average length of 9K tokens) designed to assess long-range temporal and causal retrieval. (2) **HotpotQA** (Yang et al., 2018): a multi-hop question answering benchmark requiring reasoning over multiple supporting facts. We use it to evaluate whether the memory retriever can identify and connect dispersed evidence across documents, thereby testing cross-evidence retrieval and compositional reasoning capability.

Baselines. We compare HAGE with Full Context and five state-of-the-art memory architectures using the same backbone LLMs:

Full Context. Feeds the entire conversation history into the LLM.

A-MEM (Xu et al., 2025). A self-evolving agent memory system.

Nemori (Nan et al., 2025). A graph-based memory with predict-calibrate episodic segmentation.

MemoryOS (Kang et al., 2025a). A hierarchical semantic memory operating system.

MAGMA (Jiang et al., 2026a). A multi-relational memory with static edge weights and heuristic traversal.

MemSkill (Zhang et al., 2026). An RL-based skill-evolving memory method.

Metrics. Our primary metric is the LLM-as-a-Judge score (Zheng et al., 2023), which evaluates semantic correctness through an instruction-tuned

Table 1: LoCoMo comparison with LLM-as-a-judge score under different methods and backbone LLMs. Higher is better. Best results are shown in **bold**, and second-best results are underlined. HAGE is our proposed method.

Method	Multi-Hop	Temporal	Open-Domain	Single-Hop	Adversarial	Overall
<i>gpt-4o-mini</i>						
Full Context	0.468	0.562	0.486	0.630	0.205	0.481
A-MEM	0.495	0.474	0.385	0.653	0.616	0.580
MemoryOS	<u>0.552</u>	0.422	<u>0.504</u>	0.674	0.428	0.553
Nemori	0.569	0.649	0.485	0.764	0.325	0.590
MAGMA	0.528	<u>0.650</u>	0.517	<u>0.776</u>	<u>0.742</u>	<u>0.700</u>
MemSkill	0.480	0.453	0.498	0.614	0.317	0.501
HAGE (ours)	0.547	0.667	0.497	0.797	0.839	0.739
<i>Qwen2.5-3B</i>						
Full Context	0.229	0.095	<u>0.335</u>	0.227	0.244	0.215
A-MEM	0.258	0.203	0.219	0.416	0.684	0.410
MemoryOS	0.285	0.212	0.194	0.341	0.229	0.280
Nemori	0.317	<u>0.450</u>	0.379	<u>0.641</u>	0.036	0.412
MAGMA	0.301	<u>0.402</u>	0.334	<u>0.576</u>	0.589	<u>0.499</u>
MemSkill	0.149	0.079	0.158	0.187	0.266	0.179
HAGE (ours)	<u>0.315</u>	0.457	<u>0.335</u>	0.657	<u>0.603</u>	0.548

model (prompt details in the appendix). We additionally report token-level F1 as supplementary lexical measures.

Evaluation Protocol. For the RL-trained components, including trainable edge features and the query router, we adopt a 5-fold cross-validation protocol at the conversation-sample level. This ensures that all queries from the same conversation sample are kept within the same split, preventing query-level leakage across training and evaluation. Each sample is evaluated exactly once by a model that has not observed it during training. We report the mean across folds.

Training Configuration. We use the same locked training configuration across all folds and select checkpoints based only on validation reward. Detailed hyperparameters are provided in Appendix B.

4.2 Overall Performance on LoCoMo

We first evaluate HAGE on LoCoMo, a long-term conversational memory benchmark. Table 1 reports the results under two backbone LLMs: gpt-4o-mini and Qwen2.5-3B. HAGE achieves the best overall performance under both backbone settings. With gpt-4o-mini, HAGE improves the overall judge score from the strongest baseline score of 0.700 to 0.739. With Qwen2.5-3B, HAGE improves the strongest baseline score from 0.499 to 0.548. These results show that HAGE provides consistent gains across both stronger and smaller backbone models.

A closer analysis shows that HAGE is partic-

ularly effective on reasoning-intensive categories. Under gpt-4o-mini, HAGE achieves the best scores on Temporal, Single-Hop, Adversarial, and Overall categories, with especially large gains on Adversarial queries. Under Qwen2.5-3B, HAGE achieves the best scores on Temporal, Single-Hop, and Overall categories. These gains suggest that learned query-adaptive traversal can help retrieve more useful evidence before answer generation, reducing the burden on the backbone LLM.

4.3 Generalization to Non-Conversational Multi-Hop QA

To evaluate whether HAGE generalizes beyond long-term conversational memory, we further evaluate it on HotpotQA under the distractor setting. Unlike LoCoMo, HotpotQA is a non-conversational multi-hop question answering benchmark that requires identifying and combining supporting evidence from multiple distractor passages. This setting provides a complementary testbed for evidence-intensive multi-hop reasoning.

As shown in Table 2, HAGE achieves the best overall performance on HotpotQA under the distractor setting, obtaining the F1 score of 0.678 and the LLM score of 0.824 with GPT-4o-mini. The same trend also holds for Qwen2.5-3B, where HAGE consistently outperforms all baselines. These results indicate that HAGE’s learned traversal mechanism can generalize beyond conversational memory and remain effective in non-conversational multi-hop reasoning settings.

Table 2: HotpotQA comparison with F1 and LLM score under the distractor setting. Higher is better. Best results are shown in **bold**, and second-best results are underlined.

Method	GPT-4o-mini		Qwen2.5-3B	
	F1	LLM Score	F1	LLM Score
A-MEM	0.433	0.547	0.186	0.416
MemoryOS	0.477	0.592	<u>0.350</u>	<u>0.459</u>
Nemori	0.131	0.624	0.091	0.332
MAGMA	<u>0.640</u>	<u>0.807</u>	0.337	0.424
MemSkill	0.579	0.779	0.179	0.247
HAGE	0.678	0.824	0.429	0.527

Table 3: Accuracy–efficiency trade-off on the LoCoMo benchmark. Average score is evaluated by LLM-as-a-Judge, while token consumption and latency measure inference-time cost. Best and second-best results are highlighted in bold and underlined, respectively.

Method	Avg. Score	Tokens/Query (K)	Latency (s)
A-MEM	0.580	<u>2.62</u>	2.26
MemoryOS	0.553	4.76	32.68
Nemori	0.590	3.46	2.59
MAGMA	<u>0.700</u>	3.37	<u>1.72</u>
MemSkill	0.501	0.92	1.46
HAGE	0.739	3.82	2.17

4.4 System Efficiency Analysis

To evaluate the system efficiency of HAGE, we focus on two deployment-time metrics: (1) average token cost per query and (2) average query latency. We also report the average task score to compare the accuracy–efficiency trade-off across methods.

Table 3 reports the accuracy–efficiency comparison across different memory methods. HAGE achieves the highest average score among all methods. This improvement comes with a moderate increase in inference cost: HAGE uses 3.82K tokens per query and reaches an average latency of 2.17s. Compared with the most efficient high-performing baseline, HAGE trades a small amount of additional token and latency overhead for a clear improvement in average score.

Overall, the results suggest that HAGE provides a favorable accuracy–efficiency trade-off. It achieves the best task performance while keeping token consumption and latency within the same order of magnitude as other retrieval-based memory methods.

4.5 Effect of Learned Edges and Routing

Table 4 analyzes the contribution of different HAGE components. Static edges achieve a Judge

Table 4: Breakdown analysis on the performance impact of different schemes in HAGE.

HAGE schemes	Judge	F1
Static Edge	0.698	0.462
LLM Scorer Edges	0.712	0.500
Trainable Edge	0.724	0.514
Trainable Router	0.713	0.502
HAGE	0.739	0.548

score of 0.698, showing that the underlying graph structure is useful but insufficient when traversal relies on fixed edge semantics. LLM-scored edges improve the score to 0.712, and trainable edges further improve it to 0.724, indicating that query-aware and learned edge representations provide stronger retrieval signals. The trainable-router variant also improves over static edges, suggesting that adaptive traversal decisions are important for selecting useful evidence.

The full HAGE model performs best across all metrics, reaching 0.739 Judge, 0.548 F1. These gains suggest that learned edge representations and trainable routing are complementary: edge learning captures query-dependent relational usefulness, while router learning determines how to exploit these relational signals during traversal. This explains why jointly optimizing both components outperforms using static edges, LLM-scored edges, or a trainable router alone.

5 Conclusion

We present HAGE, a weighted multi-relational memory framework that formulates agentic memory retrieval as query-conditioned traversal over dynamic relational graphs. By coupling relation-aware graph traversal with reinforcement learning-based optimization of routing policies and edge representations, HAGE enables memory retrieval to adapt to both query intent and downstream task feedback. Empirical results show that HAGE improves long-horizon reasoning accuracy and offers a favorable accuracy-efficiency trade-off compared to state-of-the-art agentic memory systems. These findings suggest that dynamic, trainable, and relation-aware memory structures offer a promising foundation for more capable LLM agents.

Limitations

HAGE has several limitations that scope the interpretation of our results.

Benchmark coverage. Our evaluation covers two benchmarks—LoCoMo (long-term conversational memory) and HotpotQA (non-conversational multi-hop QA). While these represent complementary retrieval settings, results may not fully generalize to other memory-intensive tasks such as procedural or document-grounded reasoning.

Dependence on LLM components. Both query analysis (relation intent extraction) and evaluation (LLM-as-a-Judge) rely on instruction-tuned LLMs. This introduces cost and model-specific variability; the accuracy of the relation intent classifier directly affects the quality of query-conditioned edge features used during traversal.

Scope of training signals. The reinforcement-learning-based optimization currently relies on node-level evidence targets derived from ground-truth answers. Extending HAGE to settings with weaker or noisier supervision (e.g., implicit user feedback) remains future work.

Ethical Considerations

Persistent memory systems inherently raise privacy concerns: agents that accumulate detailed user interaction histories may retain sensitive personal information beyond its intended scope. In personalized agent deployments, this could enable misuse if memory stores are accessed without user consent or appropriate safeguards. Additionally, RL-optimized retrieval policies may learn to surface information in ways that reflect biases present in training data. We encourage practitioners deploying memory-augmented agents to implement appropriate data retention policies and user-control mechanisms. On the positive side, HAGE contributes to the development of more capable long-horizon AI agents by enabling structured, relation-aware memory retrieval, with potential benefits for applications such as personal assistants, knowledge-intensive dialogue systems, and automated research agents.

All datasets and models used in this work are publicly available and used in accordance with their respective licenses (LoCoMo under CC BY-NC 4.0; HotpotQA under CC BY-SA 4.0; all-MiniLM-L6-v2 under Apache 2.0; GPT-4o-mini via the OpenAI API; Qwen2.5-3B under the Qwen Research License). No new datasets are introduced.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, and 1 others. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Petr Anokhin, Nikita Semenov, Artyom Sorokin, Dmitry Evseev, Andrey Kravchenko, Mikhail Burtsev, and Evgeny Burnaev. 2024. Arigraph: Learning knowledge graph world models with episodic memory for llm agents. *arXiv preprint arXiv:2407.04363*.
- Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020a. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.
- Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020b. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, and 1 others. 2022. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020a. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020b. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.
- Yucan Guo, Miao Su, Saiping Guan, Zihao Sun, Xiaolong Jin, Jiafeng Guo, and Xueqi Cheng. 2025. Routerag: Efficient retrieval-augmented generation from text and graph via reinforcement learning. *arXiv preprint arXiv:2512.09487*.
- Bernal J Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in neural information processing systems*, 37:59532–59569.

- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, and 1 others. 2025. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564*.
- Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, and 1 others. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*.
- Zixia Jia, Jiaqi Li, Yipeng Kang, Yuxuan Wang, Tong Wu, Quansen Wang, Xiaobo Wang, Shuyi Zhang, Junzhe Shen, Qing Li, and 1 others. 2026. The ai hippocampus: How far are we from human memory? *arXiv preprint arXiv:2601.09113*.
- Dongming Jiang, Yi Li, Guanpeng Li, and Bingzhe Li. 2026a. Magma: A multi-graph based agentic memory architecture for ai agents. *arXiv preprint arXiv:2601.03236*.
- Dongming Jiang, Yi Li, Songtao Wei, Jinxin Yang, Ayushi Kishore, Alysa Zhao, Dingyi Kang, Xu Hu, Feng Chen, Qiannan Li, and 1 others. 2026b. Anatomy of agentic memory: Taxonomy and empirical analysis of evaluation and system limitations. *arXiv preprint arXiv:2602.19320*.
- Wenqi Jiang, Suvinay Subramanian, Cat Graves, Gustavo Alonso, Amir Yazdanbakhsh, and Vidushi Dadu. 2025. Rago: Systematic performance optimization for retrieval-augmented generation serving. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, pages 974–989.
- Ziyan Jiang, Xueguang Ma, and Wenhu Chen. 2024. Longrag: Enhancing retrieval-augmented generation with long-context llms. *arXiv preprint arXiv:2406.15319*.
- Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025a. Memory os of ai agent. *arXiv preprint arXiv:2506.06326*.
- Jikun Kang, Wenqi Wu, Filippas Christianos, Alex J Chan, Fraser Greenlee, George Thomas, Marvin Purtorab, and Andy Toulis. 2025b. Lm2: Large memory models. *arXiv preprint arXiv:2502.06049*.
- Emre Kiciman, Robert Ness, Amit Sharma, and Chenhao Tan. 2023. Causal reasoning and large language models: Opening a new frontier for causality. *Transactions on Machine Learning Research*.
- Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474.
- Yi Li, Lianjie Cao, Faraz Ahmed, Puneet Sharma, and Bingzhe Li. 2026. Hippocampus: An efficient and scalable memory module for agentic ai. *arXiv preprint arXiv:2602.13594*.
- Lei Liu, Xiaoyan Yang, Yue Shen, Binbin Hu, Zhiqiang Zhang, Jinjie Gu, and Guannan Zhang. 2023. Think-in-memory: Recalling and post-thinking enable llms with long-term memory. *arXiv preprint arXiv:2311.08719*.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paran-jape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*.
- Luca Mariot, Federico Mazzone, Luca Manzoni, and Alberto Leporati. 2026. How to reconstruct (anonymously) a secret cellular automaton. *arXiv preprint arXiv:2604.11362*.
- Jiayan Nan, Wenquan Ma, Wenlong Wu, and Yize Chen. 2025. Nemori: Self-organizing agent memory inspired by cognitive science. *arXiv preprint arXiv:2508.03341*.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. *Memgpt: Towards llms as operating systems*. Preprint, arXiv:2310.08560.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22.
- Ofir Press, Noah A Smith, and Mike Lewis. 2021. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*.
- Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Defu Lian, Zhicheng Dou, and Tiejun Huang. 2025. Memorag: Boosting long context processing with global memory-enhanced retrieval augmentation. In *Proceedings of the ACM on Web Conference 2025*, pages 2366–2377.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*.
- Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th*

- international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 3982–3992.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652.
- Zheng Wang, Zhongyang Li, Zeren Jiang, Dandan Tu, and Wei Shi. 2024a. Crafting personalized agents through retrieval-augmented generation on editable memory graphs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 4891–4906.
- Zheng Wang, Shu Teo, Jieer Ouyang, Yongjun Xu, and Wei Shi. 2024b. M-rag: Reinforcing large language model performance through retrieval-augmented generation with multiple partitions. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1966–1978.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022a. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022b. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*.
- Zhaofen Wu, Hanrong Zhang, Fulin Lin, Wujiang Xu, Xinran Xu, Yankai Chen, Henry Peng Zou, Shaowen Chen, Weizhi Zhang, Xue Liu, and 1 others. 2026. Gam: Hierarchical graph-based agentic memory for llm agents. *arXiv preprint arXiv:2604.12285*.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Chang Yang, Chuang Zhou, Yilin Xiao, Su Dong, Luyao Zhuang, Yujing Zhang, Zhu Wang, Zijin Hong, Zheng Yuan, Zhishang Xiang, and 1 others. 2026a. Graph-based agent memory: Taxonomy, techniques, and applications. *arXiv preprint arXiv:2602.05665*.
- Xiaofang Yang, Lijun Li, Heng Zhou, Tong Zhu, Xiaoye Qu, Yuchen Fan, Qianshan Wei, Rui Ye, Li Kang, Yiran Qin, and 1 others. 2026b. Toward efficient agents: Memory, tool learning, and planning. *arXiv preprint arXiv:2601.14192*.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2369–2380.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- Chuanyue Yu, Kuo Zhao, Yuhao Li, Heng Chang, Mingjian Feng, Xiangzhe Jiang, Yufei Sun, Jia Li, Yuzhi Zhang, Qingyun Sun, and 1 others. 2026a. Graphrag-r1: Graph retrieval-augmented generation with process-constrained reinforcement learning. In *Proceedings of the ACM Web Conference 2026*, pages 1398–1409.
- Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. 2026b. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. *arXiv preprint arXiv:2601.01885*.
- Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026. Memskill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, and 1 others. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623.
- Wanjuan Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 19724–19731.

A Related Work

Recent surveys have characterized agentic memory from complementary perspectives, including brain-inspired memory taxonomies, forms–functions–dynamics frameworks, efficiency-oriented agent design, graph-based memory lifecycles, and empirical analyses of evaluation and system limitations (Jia et al., 2026; Hu et al., 2025; Yang et al., 2026b,a; Jiang et al., 2026b). We organize related work along four axes that situate HAGE

within this broader literature: context-window extension, retrieval-augmented generation, structured and graph-based agent memory, and learning memory access policies.

Context-Window Extension. A direct line of work extends the effective context length of Transformers through modified attention or positional encodings (Beltagy et al., 2020b; Press et al., 2021). More recent efforts augment decoders with auxiliary memory modules (Kang et al., 2025b) or global-memory-enhanced retrieval pipelines (Qian et al., 2025) to handle inputs that exceed even extended context windows. While these approaches mitigate the context-length bottleneck, they do not address the continual, multi-session write-back nature of agentic memory, where the memory store itself must evolve in response to new interactions.

Retrieval-Augmented Generation. RAG (Lewis et al., 2020) conditions generation on passages retrieved from a static external corpus. Subsequent work has extended this paradigm to long-context LLMs (Jiang et al., 2024), multi-partition retrieval (Wang et al., 2024b), and optimized retrieval serving (Jiang et al., 2025). Classical RAG formulations typically assume a relatively static knowledge base and retrieval over externally provided documents, even though later extensions introduce iterative or multi-hop retrieval. Agentic settings require memory that is continuously updated and accessed through multi-hop reasoning chains—motivating the shift to Memory-Augmented Generation (MAG) systems that support dynamic read–write–update loops.

Structured and Graph-Based Agent Memory. Beyond flat vector stores, a growing body of work organizes agent memory into structured representations to support richer reasoning. MemGPT (Packer et al., 2024) introduces an OS-style memory hierarchy with explicit paging. MemoryBank (Zhong et al., 2024) and Nemori (Nan et al., 2025) focus on episodic memory construction with selective write-back. A-MEM (Xu et al., 2025) adopts a Zettelkasten-inspired linking strategy for note-like memory units. MemoryOS (Kang et al., 2025a), Zep (Rasmussen et al., 2025), and Hippocampus (Li et al., 2026) propose persistent or scalable memory modules for multi-session agents. These systems improve memory persistence and organization, but many still retrieve memories through vector similarity, recency, salience, or manually specified control rules.

Graph-based memory architectures explicitly encode relational structure. GraphRAG (Edge et al., 2024) builds entity-centric community graphs for global question answering over large corpora. Ari-Graph (Anokhin et al., 2024) constructs knowledge-graph world models with evolving episodic structure, enabling relational reasoning for LLM agents. GAM (Wu et al., 2026) proposes a hierarchical graph memory organized around Event-Nodes and Episode-Nodes, demonstrating that multi-level graph organization improves long-horizon retrieval. EMG (Wang et al., 2024a) combines editable graph-structured memory with retrieval-augmented generation for personalized agents. While these systems design expressive relational memory structures, their retrieval mechanisms remain largely static—relying on fixed edge weights, type-level scoring heuristics, or single-shot similarity search—rather than learning to route queries dynamically.

Learning Memory Access Policies. A smaller but growing body of work frames memory access as a learnable decision process rather than a fixed retrieval procedure. AgeMem (Yu et al., 2026b) proposes a unified long- and short-term memory management framework trained with reinforcement learning to optimize memory operations end-to-end, demonstrating that downstream reward signals can guide when and what to retrieve. Mariot et al. (Mariot et al., 2026) reconceptualize memory access as an iterative, multi-step reconstruction process rather than a static lookup, arguing that relevant memory must often be assembled across multiple retrieval steps before it can inform generation. These works share HAGE’s core motivation—that retrieval should be optimized rather than hand-designed—but differ in scope: they focus on memory management policies or flat retrieval, whereas HAGE specifically targets query-conditioned traversal over multi-relational graph structures with jointly trained edge representations and routing policies.

Taken together, prior work demonstrates steady progress in structuring agent memory and improving retrieval coverage. HAGE addresses the intersection of these threads: structured multi-relational graph memory combined with RL-based, query-conditioned routing that adapts both traversal behavior and edge representations to downstream task feedback.

B Implementation Details

Each sample graph stores node embeddings of size $N \times 384$ using all-MiniLM-L6-v2 (Reimers and Gurevych, 2019), edge indices in COO format, integer edge-type labels, and an $E \times 4$ edge feature matrix. Training uses 5-fold cross-validation at the sample level, 20% held out for validation, and the remaining 10% strictly reserved as an unseen test set per fold. Each fold trains for 200 epochs with Adam (Kingma and Ba, 2014), using $\eta_{router} = 10^{-3}$ and $\eta_{edge} = 10^{-4}$.

The remaining hyperparameters are: discount factor $\gamma = 0.99$, baseline decay $\beta = 0.99$, anchor regularization $\lambda_{anchor} = 1.0$, hop budget $H_{max} = 5$, hit reward $R_{hit} = 10.0$, step penalty $\lambda_{step} = 0.05$, and timeout penalty $\lambda_{timeout} = 1.0$.

C Prompt Library

HAGE employs a sophisticated prompt strategy with three distinct types, each optimized for specific cognitive tasks within the memory pipeline.

C.1 Structured Event Extraction Prompt

We use a structured event extraction prompt to convert raw conversational turns into graph-compatible memory units. The prompt asks the model to identify salient entities, topics, relationships, temporal cues, and concise factual summaries. Instead of relying on free-form generation, the extractor returns a lightweight structured output that can be directly consumed by the memory construction pipeline.

Prompt Template: Event Extraction

Role: Extract structured memory information from conversational text.

Input:

- Speaker: {speaker}
- Text: {text}
- Optional context: {prev_summary}

Output fields:

- Entities or concepts mentioned in the text.
- Main topic or theme of the utterance.
- Relevant relationships among entities or events.
- Key factual information to preserve in memory.
- Temporal expressions, if any.
- A short summary with speaker attribution.

C.2 Query-Adaptive QA Prompt

During answer generation, the retrieved memory context is provided to a QA prompt together with the user query. The prompt is adapted according to the query type predicted by our router, allowing the system to emphasize different reasoning

behaviors when needed. This design keeps the generation stage grounded in retrieved memory while allowing lightweight query-specific control without exposing task-specific prompt details.

Prompt Template: Query-Adaptive QA

Role: Answer the user query using the retrieved memory context.

Input:

- Retrieved context: {context}
- User question: {question}
- Query guidance: {router_generated_instruction}

General instructions:

- Ground the answer in the provided context.
- Return a concise answer when the query asks for a specific fact.
- State that the information is unavailable when the context does not support an answer.
- Follow the router-generated guidance when additional reasoning control is required.

Answer:

C.3 Evaluation Prompt (LLM-as-a-Judge)

To ensure rigorous evaluation beyond simple n-gram overlapping, we employ a semantic scoring mechanism. The Judge LLM evaluates the alignment between the generated response and the ground truth using the following schema.

System Prompt: Semantic Grader

You are an expert evaluator assessing the semantic fidelity of a memory retrieval system. Score the Candidate Answer against the Gold Reference on a continuous scale [0.0, 1.0].

Scoring Rubric:

- **1.0 (Exact Alignment):** Captures all key entities, temporal markers, and causal relationships. Semantically equivalent.
- **0.8 (Substantially Correct):** Main point is accurate but lacks minor nuances or secondary details.
- **0.6 (Partial Match):** Contains valid information but misses key constraints (e.g., wrong date but correct event).
- **0.4 (Tangential):** Touches on the topic but misses the core information requirement.
- **0.2 (Incoherent):** Factually incorrect with only minimal topical overlap.
- **0.0 (Contradiction/Hallucination):** Completely unrelated or contradicts the ground truth.

Evaluation Constraints:

1. **Temporal Flexibility:** Accept relative time references (e.g., “next Tuesday”) if they resolve to the same period as the Gold Reference.
2. **Semantic Equivalence:** Prioritize informational content over lexical matching.
3. **Adversarial Handling:** If the Gold Reference states “Unanswerable”, the Candidate MUST explicitly state lack of information. Any hallucinated fact results in 0.0.

Input: Question: {question} | Gold: {gold} | Candidate: {generated}

Output: JSON{"score": float, "reasoning": "concise explanation"}

D Baseline Configurations

To ensure a fair and rigorous comparison, we standardized the experimental environment across all systems. Specifically, we adhered to the following protocols:

- **Full Context Baseline:** We implemented a “Full Context” baseline where the entire available conversation history is fed directly into the LLM’s context window (up to the 128k token limit of gpt-4o-mini). This serves as a “brute-force” reference to evaluate the model’s native long-context capabilities without external retrieval mechanisms.
- **Retrieval-Based Baselines:** For all baseline systems (e.g., AMem, Nemori, MemoryOS), we applied their official default hyperparameters and storage settings to reflect their standard out-of-the-box performance.
- **Unified Backbone Model:** To eliminate performance variance caused by different foundation models, all systems utilized OpenAI’s gpt-4o-mini for both retrieval reasoning and response generation.
- **Unified Evaluation:** All system outputs were evaluated using the identical *LLM-as-a-Judge* framework (also powered by gpt-4o-mini with temperature=0.0), as detailed in Appendix C.

Dataset Statistics. We conducted a comprehensive evaluation on the full LoCoMo benchmark, testing across all five cognitive categories to assess varying levels of retrieval complexity. The detailed distribution of query types is presented in Table 5.

Table 5: Distribution of query categories in the LoCoMo benchmark used for evaluation.

Query Category	Count
Single-Hop Retrieval	841
Adversarial	446
Temporal Reasoning	321
Multi-Hop Reasoning	282
Open Domain	96
Total Samples	1,986

E Dataset and Model Licenses

We use the following publicly available datasets and models in our experiments:

- **LoCoMo** (Maharana et al., 2024): Released under CC BY-NC 4.0.
- **HotpotQA** (Yang et al., 2018): Released under CC BY-SA 4.0.
- **all-MiniLM-L6-v2** (Reimers and Gurevych, 2019): Released under Apache License 2.0.
- **GPT-4o-mini** (Hurst et al., 2024): Accessed via the OpenAI API under OpenAI’s Terms of Service.
- **Qwen2.5-3B** (Yang et al., 2025): Released under the Qwen Research License Agreement.

All datasets are used for research purposes consistent with their respective licenses. No new datasets are introduced in this work.